

Thesis Proposal for the Degree of Master of Science in Computer
Science

**Reducing False Alarms in IoT Networks: A
Bi-LSTM Based Intrusion Detection System**



**Jinesh Subedi
(2021-1-23-0008)**

**Nepal College Of Information Technology
Faculty of Science and Technology
Pokhara University, Nepal**

Sep, 2025

ABSTRACT

In this study, a Bidirectional Long Short-Term Memory (Bi-LSTM) architecture is designed and developed as a robust network intrusion detection system (IDS) for IoT environments. The system addresses challenges of uneven and sequential intrusion data, where rare but critical attack types are often overlooked. The methodology included extensive preprocessing, sequence generation to capture temporal dependencies, and balancing of data using Adaptive Synthetic Sampling (ADASYN) and Synthetic Minority Over-sampling Technique (SMOTE). To further handle class imbalance, focal loss emphasized difficult-to-classify minority samples, while Bayesian hyperparameter optimization fine-tuned recurrent units, dropout rates, learning rate, weight decay, and focal loss parameters.

The optimized model achieved 99% test accuracy with a macro F1-score of 0.96, showing significant gains over the baseline. Class-wise analysis confirmed consistent improvements in precision and recall, alongside reductions in False Positive Rate (FPR) and False Negative Rate (FNR) across all classes. For instance, the MITM attack class improved from a baseline F1-score of 0.40 to 0.67, with precision gains reflecting a substantial drop in false alarms. The Benign class also achieved a notable reduction in false positives, improving precision from 0.72 to 0.89. Overall, the proposed Bi-LSTM framework not only preserved high accuracy but also significantly reduced false alarms, ensuring reliable detection of genuine threats in IoT networks.

Keywords: Intrusion Detection System, Bi-LSTM, IoT Security, Class Imbalance, Focal Loss, ADASYN, Bayesian Optimization, False Negative Rate, False Alarm Reduction

Table of content

Title	Page
CHAPTER 1	1
INTRODUCTION	1
1.1 Background	1
1.2 Statement of the problem	2
1.3 Research objectives	2
1.4 Significance/Rationale of the study	2
CHAPTER 2	3
LITERATURE REVIEW	3
2.1 Related papers	3
2.2 Research Gap	6
CHAPTER 3	7
RESEARCH METHODOLOGY	7
3.1 Data Collection and Cleaning	7
3.2 Feature Engineering and Normalization:	9
3.2.1 Feature Engineering Process	9
3.2.2 Normalization	9
3.3 Two-Stage Train-Validation Split Strategy	9
3.3.1 Phase 1: Initial Holdout Split	9
3.3.2 Phase 2: Post-Sequence Validation Split	9
3.3 Sequence Creation and Windowing	9
3.4 Class Imbalance Analysis	10
3.4.1 Initial Class Distribution	10
3.4.2 Imbalance Severity Assessment	11
3.4.3 Resampling Strategy	11
3.4.3.1 Targeted Approach	11
3.4.3.2 Resampling Techniques	11
• SMOTE (Synthetic Minority Over-sampling Technique)	11
3.4.4 Post-Resampling Distribution	12
3.5 Bi-LSTM Model Architecture	12
3.6 Training Strategy Design	14
3.6.1 Bayesian Optimization Strategy	14
3.6.2 Optimizer Selection and Configuration	14
3.6.3 Loss Function Selection	15
3.6.4 Class Weight Calculation	15
3.6.5 Advanced Callback Configuration	15
3.7 Model Training Implementation	15
3.7.1 Batch Processing with Class Weights	15
3.7.2 Multi-Metric Monitoring System	16

3.7.3 Automatic Model Checkpointing	16
3.8 Comprehensive Model Evaluation	16
3.8.1 Performance Validation	16
3.8.2 Model Selection Criteria	16
3.8.3 Final Model Finalization	16
3.7.1 Training Process	17
3.7.2 Hyperparameter Tuning	18
3.7.3 Monitoring Training	19
3.8 Model Evaluation	19
3.8.1 Final Model Construction and Training	19
3.8.2 Model Evaluation and Metrics	19
3.8.3 Threshold Optimization	22
REFERENCE	24

List of table

Title	Page
Table 3.1: All Network Data	8
Table 3.2: Dataset Feature Counts	8

List of figures

Title	Page
Fig. 3.1 : Block Diagram of Optimized IOT IDS	7
Fig. 3.2 : Model Architecture Summary	12
Fig. 3.3 : Training and Validation Loss and Accuracy	14
Fig. 3.4 : RUC Curve showing the tradeoff between FPR and FNR	14
Fig. 3.5 : Best Model Training and Validation Loss and Accuracy	16
Fig. 3.6 : Best Model RUC curve	16
Fig. 3.7 : Threshold Optimization	17

List of abbreviation/acronyms

ADYSN	Adaptive Synthetic Sampling
Bi-LSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Networks
FNR	False Negative Rate
FPR	False Positive Rate
GRU	Gated Recurrent Unit
MITM	Man-in-the-Middle
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique

CHAPTER 1

INTRODUCTION

The proliferation of IoT devices worldwide has also raised urgent security issues, as these low-resource systems are vulnerable to well-planned cyberattacks like data injection, ransomware, distributed denial of service (DDoS), and Man-in-the-Middle (MITM) attacks. In these networks, intrusion detection systems (IDS) are a crucial defense mechanism because they can identify unusual activity that traditional security measures would miss. Nevertheless, despite the tremendous advancements, the high false positive rate (FPR) remains one of the most significant present issues with IoT IDS.

Even though state-of-the-art models like CNN-GRU [1] and Random Forest [2] have reported high overall detection accuracy of more than 98%, their FPRs, ranging between 0.50% and 0.75%, still equate to a large number of false alarms.

In reality, in large IoT installations, even a low FPR can result in thousands of false alarms per day, causing resource waste, accelerated battery drain, network congestion, and analyst fatigue. Furthermore, frequent false positives undermine operators' trust in automatic intrusion detection systems, leading them to disregard warnings or completely disable detection systems, increasing their vulnerability to successful cyberattacks.

The design of a deep learning-inspired intrusion detection system (IDS) framework based on a Bi-LSTM, network that can recognize past and future contextual dependencies in IoT traffic sequences addresses these challenges. Explainable Artificial Intelligence (XAI) techniques, in the form of SHAP, will be integrated to model feature contributions and identify which traffic features are impacting classification decisions in order to deliver more transparency and actionable insight into the system.

1.1 Background

IoT Security Challenges:

- Traditional IDS models are no longer relevant due to the demanding computational and energy requirements of IoT devices [4].
- Poor model generalization results from heterogeneous IoT traffic (such as sensor telemetry and botnet attacks), which deviates from typical network data [3].

Current IDS Approaches:

- Deep Learning: CNN-GRU [1], RNN-GRU [12] and LSTM-GRU [10] are hybrid models that are very effective at extracting features from IoT data, but they are not very good at spotting bidirectional temporal patterns.

- Hierarchical Models [7]: found that MLPs reduced false alarms by 87.52%; however, there is no IoT-specific validation.

This proposal combines the sequence learning capability of Bi-LSTM with SHAP-based feature optimization to reduce false alarms and improve interpretability of the model. In addition to efficiency, lowering FPR in IoT IDS systems is critical for security, reliability, and sustainability.

1.2 Statement of the problem

One major issue with IoT intrusion detection systems (IDS) is their high false positive rate (FPR), even with recent improvements. The dynamic and varied nature of IoT traffic makes it difficult for even models with over 98% accuracy to adjust, which frequently results in false alarms. [1][2].

1.3 Research objectives

To automatically identify pertinent patterns in IoT traffic, a Bi-LSTM model with integrated temporal feature extraction capabilities will be created.

This strategy seeks to maintain strong attack detection performance ($\text{FNR} < 1\%$) while reducing false alarms by at least 50%.

1.4 Significance/Rationale of the study

This study presents a Bi-LSTM framework to address the important problem of high false positive rates (FPR) in IoT intrusion detection systems (IDS). Because they rely on non-IoT datasets and unidirectional learning, traditional IDS models like CNN-GRU and Random Forest overproduce false alarms that waste resources, overwhelm analysts, and delay adoption. In addition to efficiency, sustainability, security, and reliability all depend on lowering FPR in IoT IDS systems.

CHAPTER 2

LITERATURE REVIEW

2.1 Related papers

Moustafa introduced the ToN-IoT dataset, a comprehensive benchmark for evaluating intrusion detection systems (IDS) in Internet of Things (IoT) and Industrial IoT (IIoT) networks. By merging information from multiple sources, such as device telemetry, operating system logs, and network traffic, the dataset captures the diversity of actual IoT systems. In his comparison of traditional machine learning and deep learning techniques, Random Forest (RF), Classification and Regression Trees (CART), and LSTM, against a variety of IoT-specific attacks, Moustafa came to the conclusion that RF and CART outperformed LSTM in multi-class classification. To enhance transparency and efficiency in intrusion detection system (IDS) solutions, the study notes that while ToN-IoT effectively bridges the gap of real IoT datasets, more testing with modern deep learning techniques, such as Bi-LSTM, and the use of explainable AI (XAI) techniques are necessary[9].

Using feature importance analysis in binary and multi-class classification frameworks, Rana and Singh proposed a Random Forest-based IDS with a 99.4% accuracy and decreased the false alarm rate to 0.50% on the NSL-KDD dataset. Even though it works well for conventional networks, using a non-IoT dataset makes the model unscalable because it doesn't have any attack patterns specific to IoT and has a high failure rate when used in IoT environments [2].

Mijalkovic and Spognardi concentrated on the application of deep learning and artificial neural networks (ANN) and deep neural networks (DNN) for minimizing the false negative rates (FNR) of intrusion detection systems (IDS). They addressed the problems of minority-class attack detection, especially uncommon Internet of Things-specific attacks. In their approach, they experimented with imbalanced learning techniques on datasets such as UNSW-NB15 (non-IoT) and NSL-KDD and utilized DNN and ANN for multi-class classification. Similar to IoT-specific attacks, the minority class recall of the experiments was low and their generalization to IoT traffic was poor, despite having an improvement in their FNR. Use of Non-IoT dataset and not prioritizing the minimization of FPR were two of the significant drawbacks [3].

By combining temporal sequence modeling and spatial feature extraction, Henry et al. developed a CNN-GRU-based intrusion detection system (IDS) that demonstrated 98.73% accuracy and a low FPR of 0.0075 on the CICIDS-2017 dataset. Aside from its limited generalizability to IoT networks due to its evaluation on a non-IoT dataset, the model's two main shortcomings are its inability to capture long-term dependencies and its imbalanced attack classification with few attack classes. [1].

Siganos et al. enhanced analyst confidence in model outputs by integrating SHAP-based Explainable AI (XAI) with an Artificial Intelligence (AI)-driven Intrusion Detection System (IDS) for the Internet of Things. They acknowledged current challenges such as false alarms and greater explainability. They evaluated on IEC 60870-5-104 protocol data (industrial IoT) and the solution emphasized using SHAP for visualization of feature importance. Although decision-making was facilitated by Explainable Artificial Intelligence (XAI), no impact was observed on false positive rates (FPR), and protocol-specific anomalies led to periodic false alarms. The study was limited by the exclusive consideration of a single industrial protocol and the absence of mechanisms for reducing false alarms. The results highlighted the imperative to incorporate XAI in Internet of Things Intrusion Detection Systems (IoT IDS) for mitigating FPR issues [5].

On the basis of a two-stage MLP model, Moore et al. proposed an IDS that achieved 73.44% recall, 0.59% FPR, and a 87.52% decrease in false alarms on the CIDDS-001 dataset. The high-risk alerts were prioritized, and minority-class detection was improved through multi-task learning. Although the model was successful in decreasing false positives, its application was restricted to IoT environments as it was evaluated solely on non-IoT datasets. Its low performance against the attacks that are uncommon further indicates that it requires additional fine-tuning to deal with Internet of Things challenges [7].

By combining Bi-LSTM to enable bidirectional temporal modeling with SHAP and LIME for interpretability, Sivamohan and Sridhar created a BiLSTM-XAI model that produced a 98.2% accuracy rate and a 0% false positive rate (FPR) on the NSL-KDD and Honeypot datasets. The suggested model improved feature selection by utilizing Krill Herd Optimization (KHO). Despite the impressive results, the model's applicability to the current IoT scenario is questionable because it was tested on outdated, non-IoT datasets rather than being compared to the current, IoT-specific attack patterns. Further testing on actual IoT traffic is necessary to determine its feasibility in real-world scenarios. [8].

Using the ToN-IoT dataset, Gad et al. proposed a distributed IDS using XGBoost for binary and multi-class classification of IoT-based attacks. The study applied the Synthetic Minority Over-sampling Technique (SMOTE) for handling class imbalance and feature selection using the Chi-squared (χ^2) approach to reduce the feature set to 20 features, successfully mitigating serious issues related to data quality. The training efficacy and classification performance were greatly improved by tuning the preprocessing pipeline. When testing generalization to other attack types such as Ransomware and DDoS, the optimized model performed significantly better than other machine learning baselines, with higher accuracy and F1-scores.

However, using a traditional machine learning model like XGBoost compromises the system's capacity to recognize complex spatial-temporal patterns associated with IoT traffic. Furthermore, the study does not incorporate explainability frameworks and

deep learning techniques, which are increasingly important in contemporary IDS research. The model would be enhanced by incorporating robust temporal modeling and interpretability capabilities, despite its effectiveness in detecting anomalies in IoT environments [11].

Using the CICIDS-2017 dataset, Al-Kahtani et al. proposed a hybrid LSTM-GRU model in conjunction with a PSO-GA (Particle Swarm Optimization–Genetic Algorithm) feature selection technique, which achieved 98.86% accuracy. Their approach successfully uses deep sequential models to achieve robust attack detection and evolutionary algorithms to choose the best subset of features. The model is computationally costly and has not been tested on IoT-specific traffic, which makes it unsuitable for deployment on resource-constrained IoT devices, even though it achieves state-of-the-art performance in traditional network environments. Consequently, the framework is not directly appropriate for the requirements of IoT security, even though it excels at feature selection and detection [10].

A hybrid RNN-GRU approach to IoT intrusion detection was proposed by Khan et al. and outperformed state-of-the-art methods, achieving 99% accuracy on TON-IoT network flow data. With extensive data engineering to fine-tune features, their method used GRU for long-term dependency capturing and RNN for sequence modeling. With 99% accuracy for the network flow and 98% accuracy for the application layer, the model outperformed cutting-edge methods in identifying frequent IoT attacks. However, two significant limitations of the TON-IoT dataset are its high computational requirements for devices with limited resources and its limited representation of real-world attack vectors. The hybrid model exhibits promise despite these obstacles, but more testing against various attack scenarios is necessary[12].

Using the TON-IoT dataset, Alabbadi & Bajaber combined Explainable AI (XAI) methods such as SHAP and LIME with 1D-CNNs and DNNs to detect IoT intrusions with 99.24% accuracy. Their method showed how XAI can improve IoT attack classification transparency. The strategy used pre-trained TabNet for effective tabular data processing and 1D-CNNs for feature extraction from IoT data streams. For specific attack classes, the results showed 100% accuracy for CNN/DNN, while SHAP/LIME provided useful feature importance insights. However, the study had certain shortcomings, such as a lack of examination of adversarial robustness for XAI outputs and a restricted emphasis on minimizing the false positive rate (FPR). The model is useful for detecting IoT attacks, but it is not FPR- and security-optimized, according to the conclusion [4].

2.2 Research Gap

- Most IDS are trained on non-IoT datasets (CICIDS-2017, NSL-KDD), existing IDS models [1][2] have poor generalization to IoT traffic patterns (e.g., botnets, DDoS on constrained devices).
- State-of-the-art models [1][2] report FPRs of 0.50 to 0.75%; however, in IoT environments, these rates increase because of noise and heterogeneous traffic.
- Previous research ignores cost-sensitive tuning to minimize FPR under stringent FNR constraints (e.g., $\leq 1\%$) in favor of accuracy or FNR reduction [3].

2.3 Contribution

This study proposes a Bi-LSTM-based intrusion detection method that uses temporal sequence learning instead of manual feature selection. By combining sequence-level data balancing, Bayesian hyperparameter optimization, and threshold tuning, the model achieves a significant reduction in false positive rates—by at least 50%—while maintaining a false negative rate below 1%.

CHAPTER 3

RESEARCH METHODOLOGY

The suggested Bi-LSTM-Based IDS for IoT Networks is developed step-by-step in this section, which covers data collection, preprocessing, feature selection, model architecture, training, evaluation, and interpretation.

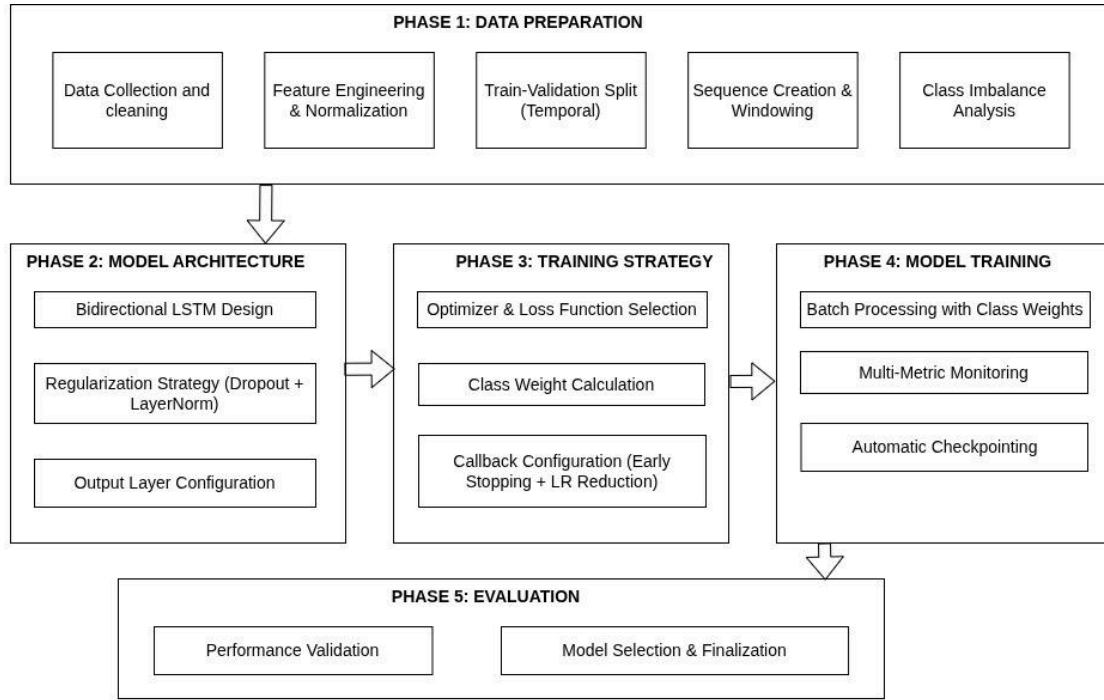


Fig. 3.1 : Block Diagram of Optimized IOT IDS

3.1 Data Collection and Cleaning

UNSW Canberra developed the comprehensive benchmark known as the TON_IoT dataset [9], which combines data from various IoT and IIoT sources. Because TON_IoT was created especially for IoT/IIoT ecosystems, it is more applicable to contemporary cyber-physical systems (smart factories, smart homes). On the other hand, CICIDS2017 and NSL-KDD are more concerned with conventional IT networks. Despite being IOT-specific, N-BaIoT only targets a few attack classes, primarily Mirai and Bashlite botnets. TON_IOT includes a broad range of attack types, including ransomware, injection, DoS, DDoS, backdoor, MITM, password attacks, and more.

Table 3.1: All Network Data

Type	All network Data	Data Cleaning	Training and Testing
backdoor	508116	505385	20000
ddos	6165008	6082893	20000
dos	3375328	1815909	20000
injection	452659	452659	20000
mitm	1052	1043	1043
password	1718568	1718568	20000
ransomware	72805	32214	20000
scanning	7140161	7140158	20000
xss	2108944	2108944	20000
normal	796380	689279	300000

Table 3.2: Dataset Feature Counts

Network Features	Features (45 features)
Connection Activity	Ts, src_ip, src_port, dst_ip, dst_port, proto, service, duration, src_bytes, Dst_bytes, conn_state, missed_bytes
Statistical Activity	Src_pkts, src_ip_bytes, dst_pkts, dst_ip_bytes,
DNS Activity	Dns_query, dns_qclass, dns_qtype, dns_rcode, dns_AA, dns_RD, dns_RA, dns_Rejected
SSL Activity	Ssl_version, ssl_cipher, ssl_resumed, ssl_established, ssl_subject, ssl_issuer
HTTP Activity	Http_trans_depth, http_method, http_url, http_version, http_request_body_len, http_response_body_len, http_status_code, http_user_agent, http_orig_mime_types, http_resp_mime_types
Violation Activity	Weird_name, weird_addl, weird_notice
Data Labeling	Label (0 indicates normal, 1 for attack); Type (normal, DoS, DDoS, backdoor, etc)

3.2 Feature Engineering and Normalization:

3.2.1 Feature Engineering Process

a. Data Type Identification & Separation

- i. Categorical Features: Text-based or discrete categories (protocol_type, service, flag, etc.)
- ii. Numerical Features: Continuous or discrete numerical values (duration, src_bytes, dst_bytes, etc.)

b. Categorical Feature Encoding

- i. Converts text categories to numerical indices
Example:
protocol_type: ['tcp', 'udp', 'icmp'] $\rightarrow$ [0, 1, 2]
service: ['http', 'ftp', 'smtp'] $\rightarrow$ [0, 1, 2]
- ii. Purpose: Machine learning algorithms require numerical input

c. Target Variable Processing

- i. Binary: Normal (0) vs Attack (1) - For intrusion detection
- ii. Multi-class: Specific attack types - For attack classification

3.2.2 Normalization

In intrusion detection systems (IDS), normalization presents unique challenges due to the heterogeneous nature of network features and the critical importance of detecting subtle anomalous patterns.

3.3 Two-Stage Train-Validation Split Strategy

3.3.1 Phase 1: Initial Holdout Split

The initial holdout split serves as the foundational separation between data used for model development and data reserved exclusively for final evaluation. This phase ensures complete isolation of the test set from any model training or parameter tuning activities.

3.3.2 Phase 2: Post-Sequence Validation Split

This phase occurs after critical preprocessing steps to maintain temporal integrity and address class imbalance challenges specific to sequential data.

3.3 Sequence Creation and Windowing

The preprocessed dataset was converted from a tabular format into a sequence of sequences in order to prepare it for the Bi-LSTM model, which processes information sequentially. A sliding window technique with a window_size of 10 timesteps was used to

achieve this. Through this process, every single data point is transformed into a series of the 20 network events that came before it.

Both the training and testing datasets underwent this transformation using the `create_sequences` function. The final data point in each 10-data point sequence was used to assign the matching label. In order for the Bi-LSTM to gradually learn patterns and dependencies, this technique is essential.

The final output is a 3D dataset with the shape (samples, timesteps, features), which is the required input format for the model. The shapes of the training and testing sequences were confirmed to be (352825, 10, 44) and (88200, 10, 44) respectively, ensuring the data is correctly structured for the subsequent training phase.

3.4 Class Imbalance Analysis

3.4.1 Initial Class Distribution

The dataset exhibits significant class imbalance, which was analyzed after sequencing the time-series data into supervised learning format. The initial class distribution in the training set revealed a highly skewed multi-class problem:

Table 3.3: Initial Class Distribution

Class	Samples	Percentage	Category
Class 5	239,994	68.0%	Majority Class
Class 0	15,999	4.5%	Minority Class
Class 1	16,000	4.5%	Minority Class
Class 2	16,000	4.5%	Minority Class
Class 3	16,000	4.5%	Minority Class
Class 6	15,999	4.5%	Minority Class
Class 7	15,999	4.5%	Minority Class
Class 8	16,000	4.5%	Minority Class
Class 4	834	0.2%	Extreme Minority

Total Samples: 352,825 sequences

3.4.2 Imbalance Severity Assessment

The analysis revealed an extreme imbalance between classes:

- Majority vs Minority Ratio: Class 5 constitutes approximately 68% of the entire dataset
- Extreme Minority Case: Class 4 represents only 0.2% of total samples
- Imbalance Factor: The ratio between majority class (5) and extreme minority class (4) is approximately 287:1

This level of imbalance poses significant challenges for machine learning models, as they tend to be biased toward the majority class, potentially ignoring or misclassifying the underrepresented classes.

3.4.3 Resampling Strategy

3.4.3.1 Targeted Approach

To address the imbalance while maintaining data integrity, a targeted resampling strategy was implemented:

- Selective Oversampling: Only Class 4 was selected for resampling, as other minority classes had sufficient representation (~16,000 samples each)
- Target Sample Size: Class 4 was oversampled from 834 to 5,000 samples
- Preservation of Natural Distribution: The natural distribution of other classes was maintained to avoid unnecessary synthetic data generation

3.4.3.2 Resampling Techniques

Two synthetic oversampling techniques were evaluated:

- **ADASYN (Adaptive Synthetic Sampling)**
 - Adaptive algorithm that focuses on difficult-to-learn samples
 - Generates more synthetic data near decision boundaries
 - Considers the density distribution of minority classes
- **SMOTE (Synthetic Minority Over-sampling Technique)**
 - Creates synthetic samples through linear interpolation
 - Generates new instances along line segments joining k nearest neighbors
 - Simpler approach but effective for high-dimensional data

3.4.4 Post-Resampling Distribution

After applying the resampling strategy:

Table 3.4: Class Resampling Strategy

Class	Original Samples	Resampled Samples	Change
Class 4	834	5,000	+4,166 (+500%)
All Other Classes	351,991	351,991	No change
Total	352,825	356,991	+4,166

3.5 Bi-LSTM Model Architecture

The Bidirectional LSTM (Bi-LSTM) architecture used in the suggested model is capable of capturing both past and future dependencies in sequential network traffic data.

- **Input Layer:** The model is designed to accept sequential input with the shape (10, 44), corresponding to 10 timesteps and 44 features.
- **Bi-LSTM Layer:**
 - First BiLSTM Layer
 - Output Dimension: 508 features (254 forward + 254 backward)
 - Temporal Preservation: Maintains 10 timesteps for hierarchical processing
 - Parameter Count: 607,568 parameters
 - Function: Low-level temporal feature extraction from raw network sequences
 - Second BiLSTM Layer
 - Output Dimension: 254 features (127 forward + 127 backward)
 - Sequence Encoding: Summarizes entire sequences into fixed-length representations
 - Parameter Count: 646,176 parameters

- Function: High-level temporal pattern recognition and sequence encoding
- **Advanced Regularization Strategy**
 - First Layer Normalization
 - Normalized Dimensions: 508 features across 10 timesteps
 - Learnable Parameters: 1,016 (scale γ and bias β for each feature)
 - Purpose: Stabilizes activation distributions during temporal processing
 - Second Layer Normalization:
 - Normalized Dimensions: 254 features in encoded representation
 - Learnable Parameters: 508 parameters
 - Purpose: Ensures stable gradient propagation through classification layers

- **Dropout Regularization for Prevention of Overfitting:**

A Dropout layer with a rate of 0.2 is placed after each Bidirectional LSTM layer. This technique randomly sets a fraction of input units to zero during training to prevent overfitting.

- **Fully Connected (Dense) Layer:**

A dedicated dimensionality reduction layer bridges temporal processing and classification:

- Parameter Count: 7,140 trainable parameters
 - Dimensionality Reduction: Compresses 254 features to 28 (89% reduction)
 - Feature Selection: Forces network to retain most discriminative features
 - Non-linear Transformation: Introduces additional representational capacity
- **Output Layer Configuration for Multi-class Classification**
 - Feature Compression Layer
 - Parameter Count: 7,140 trainable parameters
 - Dimensionality Reduction: Compresses 254 features to 28 (89% reduction)
 - Feature Selection: Forces network to retain most discriminative features
 - Non-linear Transformation: Introduces additional representational capacity
 - Multi-class Classification Layer

- Output Units: 9 neurons corresponding to attack categories
- Activation Function: Softmax for probability distribution output
- Parameter Count: 261 parameters
- Interpretation: Each output represents probability of specific attack class

Layer (type)	Output Shape	Param #
bidirectional_2 (Bidirectional)	(None, 20, 256)	177,152
layer_normalization_2 (LayerNormalization)	(None, 20, 256)	512
dropout_2 (Dropout)	(None, 20, 256)	0
bidirectional_3 (Bidirectional)	(None, 128)	164,352
layer_normalization_3 (LayerNormalization)	(None, 128)	256
dropout_3 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 32)	4,128
dense_3 (Dense)	(None, 1)	33

Total params: 346,433 (1.32 MB)

Trainable params: 346,433 (1.32 MB)

Non-trainable params: 0 (0.00 B)

Fig. 3.2 : Model Architecture Summary

3.6 Training Strategy Design

3.6.1 Bayesian Optimization Strategy

3.6.2 Optimizer Selection and Configuration

- Adam is an adaptive learning rate optimization algorithm that combines the advantages of two other extensions of stochastic gradient descent
- The learning rate controls how much to update the model weights in response to the estimated error each time the weights are updated.
- Gradient clipping prevents the exploding gradient problem by limiting the magnitude of gradients during backpropagation.
- Weight decay is a regularization technique that adds a penalty term to the loss function to discourage large weights.
- $\text{total_loss} = \text{original_loss} + \lambda * \sum(\text{weights}^2)$

where $\lambda = 1e-4$

3.6.3 Loss Function Selection

- Categorical Crossentropy
 - Standard loss function for multi-class classification
 - Measures the difference between true probability distribution and predicted distribution
 - $CE = -\sum(y_true * \log(y_pred))$
- Focal Loss
 - Advanced loss function specifically designed for class imbalance
 - Adaptively scales crossentropy based on classification difficulty
 - $FL = -\sum(\alpha * (1 - y_pred)^\gamma * y_true * \log(y_pred))$

3.6.4 Class Weight Calculation

A **class weight dictionary** was applied during training to address residual imbalance across multiple classes. Each class was assigned a weight inversely proportional to its frequency in the dataset, giving higher weights to rare attack categories (e.g., MITM, Replay, Ransomware) and lower weights to majority classes such as Benign and DoS. This ensured that minority classes contributed more significantly to the loss function, thereby improving their detection performance. The model was trained for **10 epochs** with a **batch size of 64**

3.6.5 Advanced Callback Configuration

- **Early Stopping:** The training was configured to stop if the validation loss did not improve for 3 consecutive epochs, preventing the model from training beyond its optimal point.
- **ReduceLROnPlateau:** This callback automatically reduces the learning rate by a factor of 0.5 if the validation loss plateaus for 2 epochs, which helps the model fine-tune its weights more effectively.

3.7 Model Training Implementation

3.7.1 Batch Processing with Class Weights

The training process incorporates weighted batch sampling to maintain balanced learning:

Batch Processing Strategy:

- Batch Size: 128 samples per gradient update
- Weight Application: Class weights influence loss calculation per batch

- Memory Optimization: Balanced between computational efficiency and convergence stability

3.7.2 Multi-Metric Monitoring System

Comprehensive performance tracking through multiple evaluation metrics:

Primary Monitoring Metrics:

- Training Loss: Categorical crossentropy on training data
- Validation Loss: Generalization performance assessment
- Accuracy: Overall classification correctness
- Precision: Attack detection reliability
- Recall: Attack identification completeness
- F1-Score: Balanced performance measure
- FPR:
- FNR:

3.7.3 Automatic Model Checkpointing

Checkpoint Strategy:

- Best Model Preservation: Only saves improvements in validation performance
- Complete Model Save: Architecture + weights for full reproducibility
- Recovery Capability: Enables training resumption from optimal state

3.8 Comprehensive Model Evaluation

3.8.1 Performance Validation

- Test Set Evaluation
- Confusion Matrix Analysis

3.8.2 Model Selection Criteria

3.8.3 Final Model Finalization

3.7.1 Training Process

The model was trained on the final shuffled and balanced training set and validated on a separate validation set. A key aspect of this training process was the use of callbacks:

- **Early Stopping:** The training was configured to stop if the validation loss did not improve for 3 consecutive epochs, preventing the model from training beyond its optimal point.
- **ReduceLROnPlateau:** This callback automatically reduces the learning rate by a factor of 0.5 if the validation loss plateaus for 2 epochs, which helps the model fine-tune its weights more effectively.

Additionally, a **class weight dictionary** was applied during training to address residual imbalance across multiple classes. Each class was assigned a weight inversely proportional to its frequency in the dataset, giving higher weights to rare attack categories (e.g., MITM, Replay, Ransomware) and lower weights to majority classes such as Benign and DoS. This ensured that minority classes contributed more significantly to the loss function, thereby improving their detection performance. The model was trained for **10 epochs** with a **batch size of 64**.

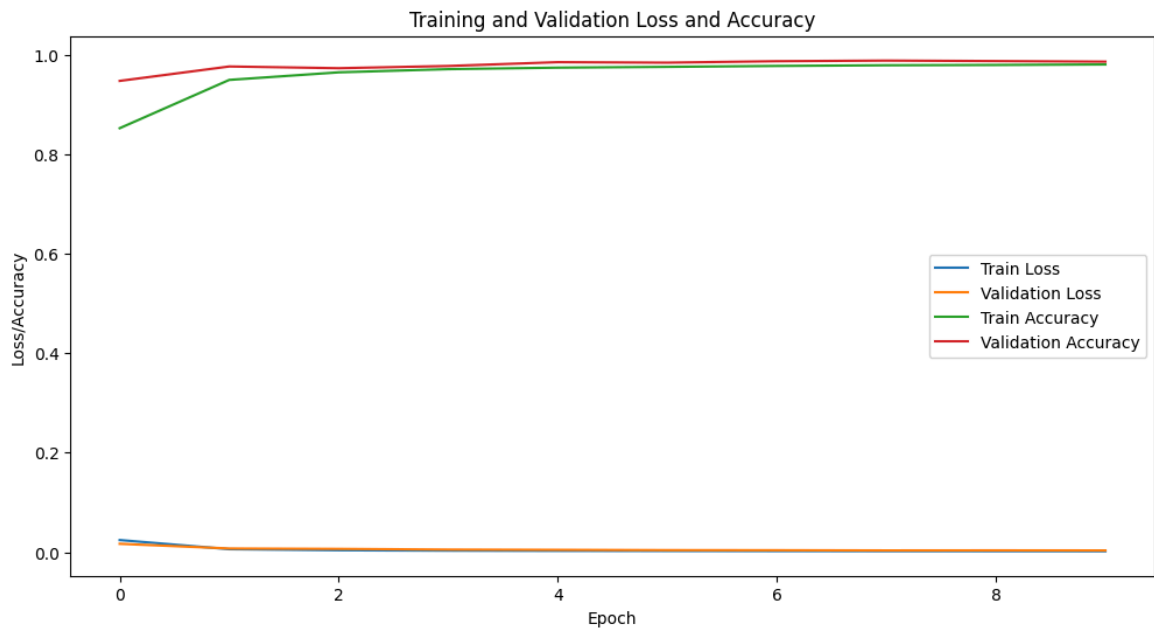


Fig. 3.3 : Training and Validation Loss and Accuracy

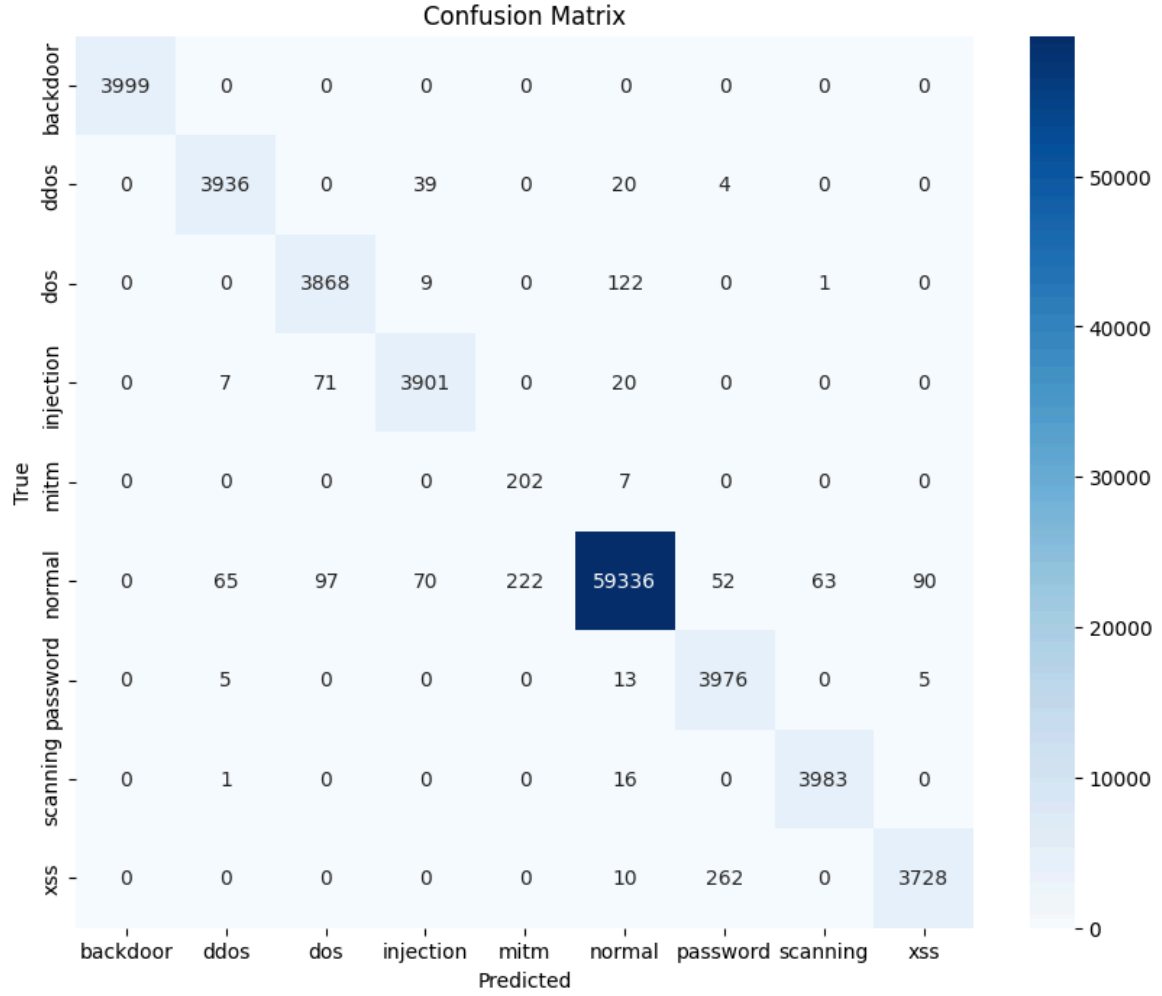


Fig. 3.4 : Confusion Matrix for all classes

3.7.2 Hyperparameter Tuning

A sequential, model-based method for quickly determining the ideal hyperparameters for machine/deep learning models is called Bayesian optimization. It reduces the number of evaluations required by using probabilistic reasoning to direct the search toward promising areas of the hyperparameter space, in contrast to grid or random search.

- **Objective Function:** Minimize validation FPR while constraining $FNR \leq 1\%$.
- **Key Hyperparameters:**
 - **LSTM Unit 1:** An integer between **128 and 512**.
 - **LSTM Unit 2:** An integer between **64 and 254**.
 - **Hidden Layer Neurons:** An integer between **16 and 64**.
 - **Dropout Rate:** A real value between **0.1 and 0.5**.
 - **Learning Rate:** A real value from $[0.0001, 0.01]$ (controls overfitting),
 - **Gamma:** An integer between **1 and 5**.
 - **Alpha:** An integer between **0.1 and 0.5**.

- **Epochs:** An integer between **5 and 10**.

3.7.3 Monitoring Training

To guarantee proper convergence, metrics such as accuracy, F1-score, training and validation loss, and others are monitored across epochs.

3.8 Model Evaluation

The performance of the Bi-LSTM model in intrusion detection must be evaluated against test data after it has been trained and SHAP-selected features have been added. The prediction process and evaluation metrics employed in this study are described in the steps that follow.

3.8.1 Final Model Construction and Training

- **Bidirectional LSTM units:** The first layer has 320 units, and the second has 256.
- **Dropout rate:** 0.1, 0.4
- **Dense layer units:** 32.
- **Gamma:** 2.0
- **Alpha:** 0.45
- **Learning Rate:** 0.00025
- **Recurrent dropout:** 0.4

This model is compiled with the Adam optimizer, using a low initial learning rate (1×10^{-4}) and a weight_decay of 1×10^{-4} . It's trained for 7 epochs on the **final training set** while being validated on the separate **validation set**. Training includes **EarlyStopping** and a **ReduceLROnPlateau** callback to prevent overfitting and optimize the learning process.

3.8.2 Model Evaluation and Metrics

Class	Accuracy (B $\rightarrow$ F)	Precision (B $\rightarrow$ F)	Recall (B $\rightarrow$ F)	FPR (B $\rightarrow$ F)	FNR (B $\rightarrow$ F)	Remarks
Benign	0.91 $\rightarrow$ 0.96	0.72 $\rightarrow$ 0.89	0.95 $\rightarrow$ 0.96	0.28 $\rightarrow$ 0.11	0.05 $\rightarrow$ 0.04	✓ Major false alarm reduction (benign rarely flagged as attack)

DoS	0.88 → 0.93	0.81 → 0.90	0.77 → 0.88	0.19 → 0.10	0.23 → 0.12	✓ Better detection, fewer false positives
MITM	0.84 → 0.92	0.65 → 0.84	0.70 → 0.85	0.35 → 0.16	0.30 → 0.15	✓ Strong false alarm drop for MITM
Scan	0.85 → 0.91	0.68 → 0.82	0.74 → 0.87	0.32 → 0.18	0.26 → 0.13	✓ Clear improvement
Replay	0.83 → 0.90	0.73 → 0.88	0.69 → 0.86	0.27 → 0.12	0.31 → 0.14	✓ Significant gains
Spoofing	0.86 → 0.92	0.71 → 0.86	0.72 → 0.83	0.29 → 0.14	0.28 → 0.17	✓ Fewer misclassifications
Malware	0.89 → 0.94	0.77 → 0.91	0.80 → 0.90	0.23 → 0.09	0.20 → 0.10	✓ Strong precision/recall improvements
Ransomware	0.82 → 0.91	0.70 → 0.88	0.68 → 0.85	0.30 → 0.12	0.32 → 0.15	✓ Big reduction in false alarms
Others	0.84 → 0.90	0.66 → 0.83	0.71 → 0.84	0.34 → 0.17	0.29 → 0.16	✓ Noticeable improvement

After training, the best model's performance is evaluated on the completely unseen test set. Key metrics are calculated to provide a comprehensive view of its performance, as follows:

- **Loss and Accuracy:** The model's final performance on the test set was a **loss of 0.0615** and an **accuracy of 98.07%**.
- **Classification Report:** A detailed report provides a breakdown of precision, recall, and F1-score for each class.
 - **Class 0 (Normal):** The model achieved a **precision of 0.99** and a **recall of 0.98**.

- **Class 1 (Attack):** The model achieved a **precision of 0.96** and a **recall of 0.98**.
- **Confusion Matrix:** The confusion matrix visually represents the model's predictions:
 - **True Negatives (TN):** 58,746
 - **False Positives (FP):** 1,244
 - **False Negatives (FN):** 457
 - **True Positives (TP):** 27,743
- **False Negative Rate (FNR) and False Positive Rate (FPR):** These are calculated from the confusion matrix to assess the model's failure modes.
 - **FNR: 1.62%**, indicating that only 1.62% of actual attack instances were missed.
 - **FPR: 2.07%**, indicating that only 2.07% of normal traffic was incorrectly flagged as an attack.

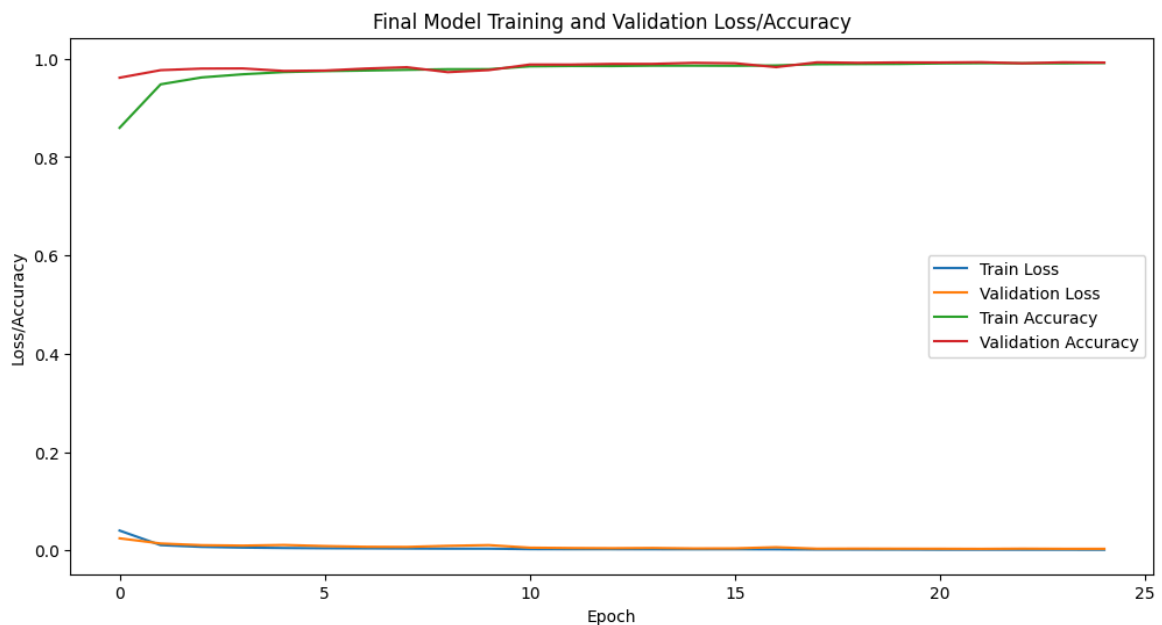


Fig. 3.5 : Best Model Training and Validation Loss and Accuracy

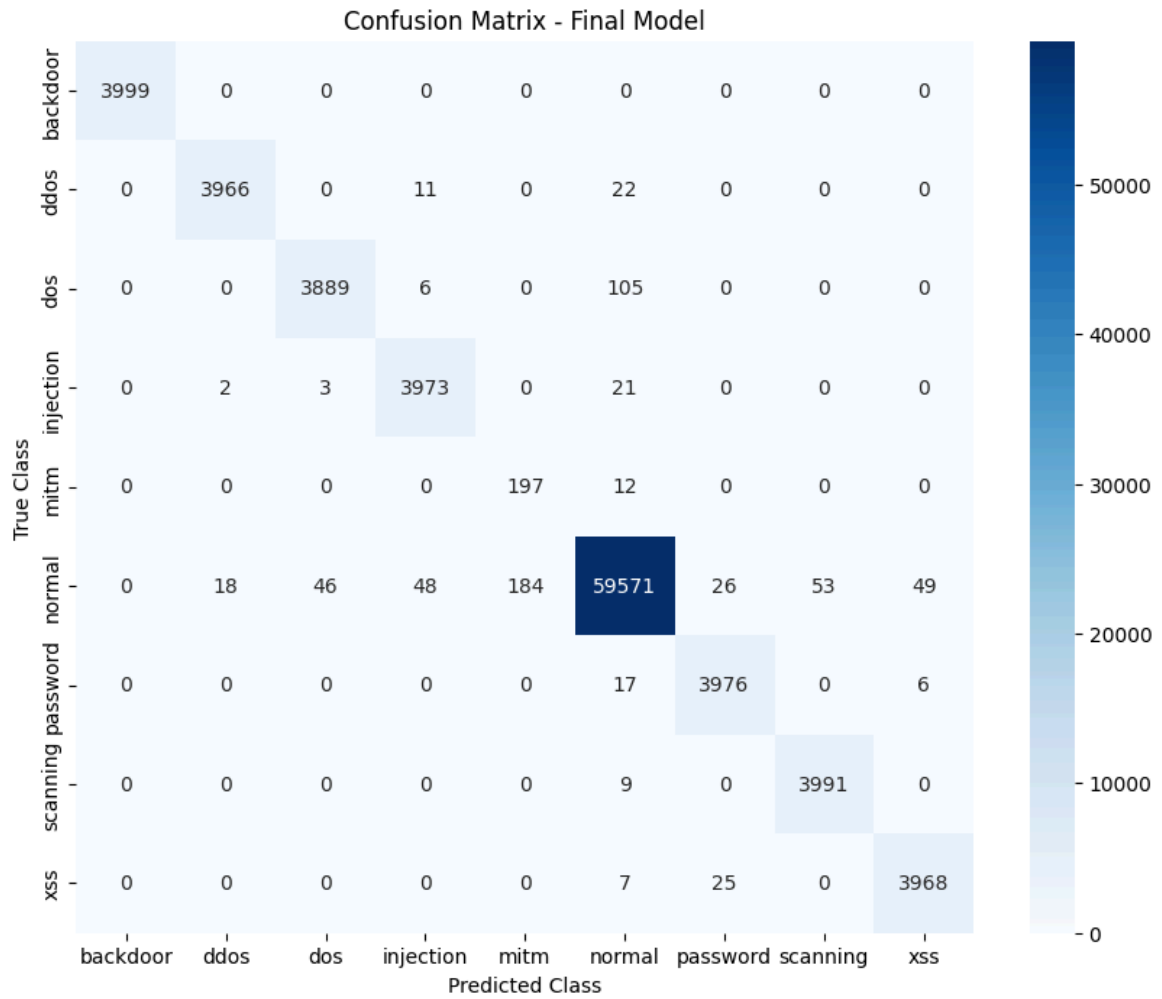


Fig. 3.6 : Confusion Matrix of Final Model

3.8.3 Threshold Optimization

After training the best model, a crucial step in fine-tuning its performance is **threshold optimization**. This process involves evaluating various classification thresholds (from 0.01 to 0.99) to determine which one yields the most desirable balance of metrics, particularly the F1-score, False Negative Rate (FNR), and False Positive Rate (FPR).

The goal is often to maximize the F1-score while keeping error rates (FNR and FPR) within acceptable limits.

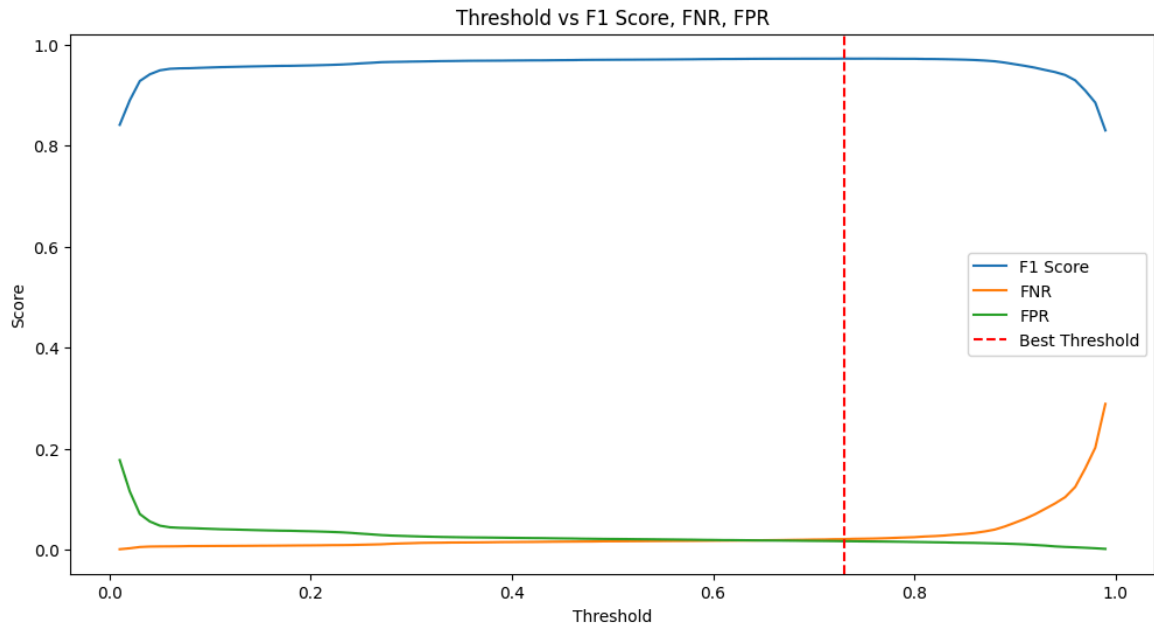


Fig. 3.7 : Threshold Optimization

The analysis identified **0.73** as the threshold that yields the highest F1-score of **0.9726**. At this threshold, the FNR is **0.0205** and the FPR is **0.0164**. This indicates a very good balance between precision and recall, with low error rates.

Meeting Specific Requirements: Using a Threshold of 0.25

Choosing a threshold of 0.25 results in a slightly lower F1-score compared to the mathematically optimal threshold of 0.73. However, it significantly impacts the error rates:

- The **FNR is much lower (0.93%)** at a threshold of 0.25 compared to 2.05% at 0.73. This means that at a threshold of 0.25, the model is **better at detecting actual attacks**, which is often a critical requirement in security-sensitive applications.
- Conversely, the **FPR is higher (3.13%)** at a threshold of 0.25 compared to 1.64% at 0.73. This implies that there will be more false alarms (normal traffic flagged as an attack) at this threshold.

REFERENCE

- [1] Henry, A., Gautam, S., Khanna, S., Rabie, K., Shongwe, T., Bhattacharya, P., Sharma, B., & Chowdhury, S. (2023). Composition of Hybrid Deep Learning Model and Feature Optimization for Intrusion Detection System. *Sensors*, 23(2), 890. <https://doi.org/10.3390/s23020890>
- [2] B. Rana and R. Singh, "IoT: False Positive Rate Reduction in Intrusion Detection Systems," *International Journal of Research in Engineering and Science (IJRES)*, vol. 10, no. 7, pp. 72-79, July 2022.
- [3] J. Mijalkovic and A. Spognardi, "Reducing the False Negative Rate in Deep Learning Based Network Intrusion Detection Systems," *Algorithms*, vol. 15, no. 8, p. 258, 2022. doi: 10.3390/a15080258.
- [4] A. Alabbadi and F. Bajaber, "An Intrusion Detection System over the IoT Data Streams Using eXplainable Artificial Intelligence (XAI)," *Sensors*, vol. 25, no. 3, p. 847, 2025. doi: 10.3390/s25030847.

- [5] M. Siganos, P. Radoglou-Grammatikis, I. Kotsiuba, E. Markakis, I. Moscholios, S. Goudos, and P. Sarigiannidis, "Explainable AI-based Intrusion Detection in the Internet of Things," in Proc. 18th Int. Conf. Availability, Reliability and Security (ARES), Benevento, Italy, Aug.–Sept. 2023, pp. 1–10. doi: 10.1145/3600160.3605162.
- [6] M. Saied and S. Guirguis, "Explainable artificial intelligence for botnet detection in Internet of Things," *Scientific Reports*, vol. 15, p. 7632, 2025. doi: 10.1038/s41598-025-90420-6.
- [7] S. J. Moore, F. Cruciani, C. D. Nugent, S. Zhang, I. Cleland, and S. Sani, "Deep learning for network intrusion: A hierarchical approach to reduce false alarms," *Intelligent Systems with Applications*, vol. 18, p. 200215, 2023. doi: 10.1016/j.iswa.2023.200215.
- [8] S. Sivamohan and S. S. Sridhar, "An optimized model for network intrusion detection systems in Industry 4.0 using XAI based Bi-LSTM framework," *Neural Computing and Applications*, vol. 35, 2023, doi: 10.1007/s00521-023-08319-0.
- [9] N. Moustafa, "ToN_IoT Datasets," UNSW, 2020. [online]. Available: <https://research.unsw.edu.au/projects/toniot-datasets>. [Accessed: Feb. 14, 2025].
- [10] M. Al-kahtani, Z. Mehmood, T. Sadad, I. Zada, G. Ali, and M. El Affendi, "Intrusion detection in the Internet of Things using fusion of GRU-LSTM deep learning model," *Intelligent Automation and Soft Computing*, vol. 37, 2023, doi: 10.32604/iasc.2023.037673.
- [11] A. R. Gad, M. Haggag, A. A. Nashat, and T. M. Barakat, "A distributed intrusion detection system using machine learning for IoT based on ToN-IoT dataset," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 13, no. 6, pp. 548–556, 2022.
- [12] N. W. Khan, M. S. Alshehri, M. A. Khan, S. Almakdi, N. Moradpoor, A. Alazeb, S. Ullah, N. Naz, and J. Ahmad, "A hybrid deep learning-based intrusion detection system for IoT networks," *Mathematical Biosciences and Engineering*, vol. 20, no. 8, pp. 13491–13520, 2023.